

GitHub Copilot Prompt — Dashboard Fix + Login/Signup

Copy the following prompt into GitHub Copilot Chat. It is designed to make Copilot inspect the existing project first, then implement authentication without rebuilding the application.

GITHUB COPILOT PROMPT — FIX DASHBOARD + ADD LOGIN/SIGNUP

I have an existing full-stack Internal Task & Management Dashboard project.

IMPORTANT:

Do not rebuild the project from scratch. First inspect the existing frontend and backend structure and understand the current implementation. Modify the existing code cleanly and preserve working features.

TECH STACK

Frontend:

- React
- Vite
- React Router
- Tailwind CSS
- JavaScript/JSX

Backend:

- Python
- FastAPI
- PostgreSQL preferred / existing database setup
- REST APIs

The project already has:

- Dashboard
- Tasks
- Task Details
- Task List
- Current user context
- Dashboard metrics
- Task assignment
- Priority/status badges
- API services
- FastAPI backend
- /api/users
- /api/tasks
- Dashboard API

CURRENT BUG

The dashboard currently shows:

"No current user found. Please make sure users are available."

The browser console shows:

Dashboard loading for user: null

No current user ID available.

When I call:

GET `http://localhost:8000/api/users`

the backend returns:

```
[]
```

The backend therefore has no users, and the frontend cannot determine the current user.

DO NOT solve this by hardcoding a user ID.

Instead, implement proper authentication with Login and Signup.

=====

MAIN OBJECTIVE

=====

Convert the current demo "acting as user" system into a proper authentication system.

Implement:

1. Signup page
2. Login page
3. Logout
4. JWT authentication
5. Password hashing
6. Current authenticated user endpoint
7. Protected dashboard/task routes
8. Authentication context on React frontend
9. Connect authentication to the existing FastAPI backend
10. Automatically associate tasks/comments/actions with the logged-in user
11. Remove the dependency on manually selecting a seeded user
12. Make the architecture easy to modify later

=====

STEP 1 — INSPECT THE EXISTING PROJECT

=====

Before changing anything, inspect:

Frontend:

- `src/App.jsx`
- `src/main.jsx`
- `src/hooks/useCurrentUser.jsx`
- `src/pages/Dashboard.jsx`
- `src/pages/TaskList.jsx`
- `src/pages/TaskDetails.jsx`
- `src/layouts/AppLayout.jsx`
- `src/services/`

- src/components/
- src/utils/

Backend:

- main.py
- database configuration
- models
- schemas
- routers/routes
- authentication-related code if present
- user APIs
- task APIs
- database initialization/migrations

Understand the existing API response formats before modifying them.

Do not duplicate existing functionality.

=====

STEP 2 — CREATE A PROPER USER MODEL

=====

If a User model does not already exist, create one.

Fields:

- id
- name
- email
- password_hash
- created_at
- updated_at

Email must be unique.

DO NOT store plain-text passwords.

Use secure password hashing with an appropriate modern library compatible with FastAPI.

=====

STEP 3 — AUTHENTICATION API

=====

Create:

POST /api/auth/signup

Request:

```
{  
  "name": "John Doe",  
  "email": "john@example.com",
```

```
"password": "password123"
}
```

POST /api/auth/login

Request:

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  }
}
```

GET /api/auth/me

Require a valid JWT and return:

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com"
}
```

POST /api/auth/logout

Implement logout appropriately for the chosen authentication strategy.

=====

STEP 4 — JWT

=====

Implement JWT authentication using a secret stored in environment variables.

DO NOT hardcode secrets.

Add to .env.example:

```
JWT_SECRET_KEY=change-this-in-production
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

Make sure .env is not committed to Git.

Create a reusable authentication dependency such as:

`get_current_user()`

It should:

1. Read the Bearer token
2. Validate the JWT
3. Extract the user ID/email
4. Fetch the user
5. Reject invalid/expired tokens
6. Return the authenticated user

=====

STEP 5 — FRONTEND AUTH CONTEXT

=====

Replace the current demo-style `CurrentUserProvider` logic.

The existing `src/hooks/useCurrentUser.jsx` currently obtains users from `GET /api/users` and selects the first user.

This is NOT what we want anymore.

Create:

`src/context/AuthContext.jsx`

or an equivalent `useAuth` hook.

It should provide:

- user
- loading
- isAuthenticated
- login()
- signup()
- logout()
- refreshUser()

Example:

```
const {
  user,
  loading,
  isAuthenticated,
  login,
  signup,
  logout
} = useAuth()
```

The logged-in user must come from:

`GET /api/auth/me`

rather than selecting a user from `/api/users`.

STEP 6 — TOKEN STORAGE

Use a reasonable authentication strategy.

Prefer secure HTTP-only cookie authentication.

If that is difficult with the existing architecture, use `localStorage` for the JWT, but centralize token handling in:

`src/utils/auth.js`

with:

- `getToken()`
- `setToken()`
- `removeToken()`

Do not access `localStorage` throughout the application.

If cookies are used, configure FastAPI CORS and cookie settings correctly.

STEP 7 — CENTRALIZED API SERVICE

Create or improve:

`src/services/api.js`

It should:

- use the backend base URL from environment variables
- automatically attach authentication
- parse JSON responses
- handle HTTP errors consistently
- handle 401 Unauthorized
- avoid repeated fetch logic

Use:

`VITE_API_URL=http://localhost:8000`

Do not hardcode the backend URL throughout the application.

STEP 8 — LOGIN PAGE

Create:

`src/pages/Login.jsx`

Keep the existing Basecamp-style UI.

Include:

- application name/logo
- email
- password
- login button
- loading state
- error message
- link to Signup

After successful login:

navigate to "/"

On failure, show a clean error message.

=====

STEP 9 — SIGNUP PAGE

=====

Create:

src/pages/Signup.jsx

Fields:

- Full name
- Email
- Password
- Confirm password

Validation:

- name required
- valid email
- password required
- reasonable minimum password length
- confirmation must match

Prefer automatically logging the user in after signup.

=====

STEP 10 — ROUTING

=====

Public:

- /login
- /signup

Protected:

- /
- /tasks
- /tasks/:id

Create:

src/components/ProtectedRoute.jsx

Unauthenticated users should be redirected to /login.

Do not duplicate authentication checks in every page.

Do not create another BrowserRouter if one already exists.

=====

STEP 11 — APP INITIALIZATION

=====

On application startup:

1. Check authentication.
2. If not authenticated, show Login.
3. If authentication exists, call GET /api/auth/me.
4. If valid, store user in AuthContext.
5. If invalid/expired, clear authentication and redirect to Login.
6. Avoid showing Dashboard before authentication state is known.

=====

STEP 12 — UPDATE DASHBOARD

=====

The Dashboard currently uses:

```
const { currentUserId } = useCurrentUser()
```

Replace this with the authenticated user.

Example:

```
const { user } = useAuth()
```

Use user.id where necessary.

Do NOT call GET /api/users just to find the current user.

=====

STEP 13 — TASK APIs

=====

Update task endpoints so authenticated identity is used where appropriate.

For sensitive operations, do not blindly trust a user ID sent from the frontend.

Use:

```
current_user = Depends(get_current_user)
```

For example:

```
GET /api/tasks?assignee=me
```

should resolve "me" using the authenticated user.

Preserve admin assignment functionality if it already exists.

STEP 14 — DASHBOARD API

Prefer:

GET /api/dashboard

with the backend obtaining the current user from JWT.

If changing the existing API would break functionality, maintain backward compatibility where reasonable.

Dashboard must display:

- Total
- Pending
- In Progress
- Completed
- Overdue
- Assigned to Me
- Upcoming tasks
- Priority breakdown
- Completion rate
- Daily tip

STEP 15 — DATABASE INITIALIZATION

The current /api/users returns [].

Do not hardcode a fake user in React.

Signup must create a real database user.

Optionally provide:

backend/seed.py

for development demo users.

Make the seed script safe to run multiple times.

The normal application must work without seeded users because Signup creates users dynamically.

STEP 16 — CORS

Configure FastAPI CORS correctly for:

Frontend:

http://localhost:5173

Backend:

http://localhost:8000

Ensure authentication requests work.

If cookies are used, configure credentials correctly.

Do not use overly permissive production CORS.

=====

STEP 17 — ERROR HANDLING

=====

Use useful HTTP status codes:

- 400/422 validation
- 401 authentication failure
- 403 authorization failure
- 404 missing resources
- 409 duplicate email
- 500 unexpected errors

Frontend messages should be friendly.

Examples:

"Invalid email or password."

"An account with this email already exists."

"Your session has expired. Please log in again."

=====

STEP 18 — LOADING STATES

=====

Every async operation must correctly handle:

- loading
- success
- error

Do not allow infinite loading.

Avoid:

if (!currentUserId) return

when loading remains true forever.

=====

STEP 19 — LOGOUT

=====

Add Logout to the existing AppLayout/header.

On logout:

1. Remove token/cookie
2. Clear AuthContext
3. Navigate to /login
4. Prevent protected pages from being accessed

STEP 20 — KEEP EXISTING UI

Do NOT redesign the dashboard.

Keep:

- colors
- typography
- Tailwind classes
- cards
- sidebar
- header
- task UI
- status badges
- priority badges

Only add authentication UI consistently.

STEP 21 — CLEAN ARCHITECTURE

Recommended structure:

frontend/src/

components/

ProtectedRoute.jsx

MetricCard.jsx

StatusBadge.jsx

PriorityBadge.jsx

context/

AuthContext.jsx

hooks/

useAuth.jsx

pages/

Login.jsx

Signup.jsx

Dashboard.jsx
TaskList.jsx
TaskDetails.jsx

services/
api.js
authService.js
dashboardService.js
taskService.js
userService.js
externalService.js

utils/
auth.js
format.js

backend/

auth/
dependencies.py
jwt.py
password.py

models/
user.py
task.py

schemas/
user.py
auth.py
task.py

routers/
auth.py
users.py
tasks.py
dashboard.py

Adapt this structure to the existing project rather than blindly creating duplicate files.

=====

STEP 22 — ENVIRONMENT VARIABLES

=====

Frontend:

VITE_API_URL=http://localhost:8000

Backend:

DATABASE_URL=...

JWT_SECRET_KEY=...

JWT_ALGORITHM=HS256

ACCESS_TOKEN_EXPIRE_MINUTES=60

Update .env.example.

Never expose backend secrets to React.

=====

STEP 23 — DO NOT BREAK EXISTING FEATURES

=====

Verify:

- Login
- Signup
- Logout
- /api/auth/me
- Dashboard
- Tasks
- Task details
- Task creation
- Task assignment
- Current user
- Protected routes
- Browser refresh
- Expired authentication
- Existing styling

=====

STEP 24 — TEST COMPLETE FLOW

=====

Test:

1. Start database.
2. Start FastAPI.
3. Start Vite.
4. Open /signup.
5. Create an account.
6. Verify user is inserted into database.
7. Verify /api/users returns the user.
8. Verify authentication token/cookie is created.
9. Verify /api/auth/me returns the user.
10. Verify Dashboard opens.
11. Verify Assigned to Me works.
12. Create a task.
13. Assign a task.

14. Verify task list.
15. Refresh browser.
16. Verify session remains active.
17. Logout.
18. Verify protected routes redirect to /login.
19. Login again.
20. Verify Dashboard loads.

=====

IMPORTANT CODING RULES

=====

- Do not hardcode user IDs.
- Do not hardcode passwords.
- Do not store plain-text passwords.
- Do not hardcode JWT secrets.
- Do not duplicate API request logic.
- Do not create multiple authentication contexts.
- Do not create another BrowserRouter.
- Do not remove working dashboard/task functionality.
- Do not rewrite unrelated components.
- Reuse existing components.
- Use async/await consistently.
- Use environment variables.
- Make database changes safely.
- Preserve existing API behavior where possible.
- Keep authentication logic reusable and maintainable.

FINAL OUTPUT REQUIRED

After implementation, provide:

1. Files created.
2. Files modified.
3. Database/model changes.
4. New npm packages.
5. New Python packages.
6. Required .env variables.
7. Commands to start frontend/backend.
8. Any migration or seed commands.
9. Any remaining issues or assumptions.

MOST IMPORTANT:

Before editing, inspect the actual project files and existing API routes. Do not assume file names, database technology, response formats, or endpoint structures. Adapt the implementation to what already exists.